acwing826. 单链表
827.双链表
828.模拟栈
3302. 表达式求值
829. 模拟队列
830. 单调栈
154 滑动窗口
831. KMP字符串
Trie树(前缀树或字典树)
    835. Trie字符串统计
    143. 最大异或对
并查集
    acwing 836. 合并集合
        cpp
        python
    837. 连通块中点的数量
        cpp
        python
    240. 食物链
    P3958 [NOIP 2017 提高组] 奶酪
堆
838. 堆排序
839. 模拟堆
哈希表
    840. 模拟散列表
        拉链法:
        开放寻址法
            cpp
            python
    AcWing 841. 字符串哈希（字符串前缀哈希法）
        cpp
        python

acwing826. 单链表

实现一个单链表，链表初始为空，支持三种操作：

1. 向链表头插入一个数；
2. 删除第 k 个插入的数后面的一个数；
3. 在第 k 个插入的数后插入一个数。

现在要对该链表进行 M 次操作，进行完所有操作后，从头到尾输出整个链表。

注意:题目中第 k 个插入的数并不是指当前链表的第 k 个数。例如操作过程中一共插入了 n 个数，则按照插入的时间顺序，这 n 个数依次为：第 1 个插入的数，第 2 个插入的数，…第 n 个插入的数。

输入格式

第一行包含整数 M，表示操作次数。

接下来 M 行，每行包含一个操作命令，操作命令可能为以下几种：

1. `H x`，表示向链表头插入一个数 x。
2. `D k`，表示删除第 k 个插入的数后面的数（当 k 为 0 时，表示删除头结点）。
3. `I k x`，表示在第 k 个插入的数后面插入一个数 x（此操作中 k 均大于 0）。

输出格式

共一行，将整个链表从头到尾输出。

数据范围

$1 \leq M \leq 100000$
所有操作保证合法。

头插法：

```
#include <iostream>
#include <cstring>
using namespace std;

const int N = 100005;
int head, e[N], ne[N], idx;
//头节点， 值，下一节点，idx索引节点的编号
void init()
{
    head = -1;
    idx = 0;
}

void add_to_head(int x  )
{
    e[idx] = x;
    ne[idx] = head;
    head = idx;
    idx ++;
}

void remove(int pos)
```

```
{
    ne[pos] = ne[ne[pos]];
}


void insert(int x, int pos)
{
    e[idx] = x;
    ne[idx] = ne[pos];
    ne[pos] = idx;
    idx++;

}
int main()
{
    int m;
    cin >> m;

    init();

    while(m--)
    {
        char op;
        int pos, x;
        cin >> op;
        if(op == 'H')
        {
            cin >> x;
            add_to_head(x);
        }
        else if(op == 'D')
        {
            cin >> pos;
            if(!pos) head = ne[head];   //移除头节点
            else remove(pos - 1);//第k个元素对应的索引为k - 1
        }
        else
        {
            cin >> pos >> x;
            insert(x, pos -1);//第k个元素对应的索引为k - 1
        }
    }
    for(int i = head; i != -1; i = ne[i]) cout << e[i] << " ";

    cout<< endl;
}


```

827.双链表

实现一个双链表，双链表初始为空，支持 5 种操作：

1. 在最左侧插入一个数；
2. 在最右侧插入一个数；
3. 将第 k 个插入的数删除；
4. 在第 k 个插入的数左侧插入一个数；
5. 在第 k 个插入的数右侧插入一个数

现在要对该链表进行 M 次操作，进行完所有操作后，从左到右输出整个链表。

注意:题目中第 k 个插入的数并不是指当前链表的第 k 个数。例如操作过程中一共插入了 n 个数，则按照插入的时间顺序，这 n 个数依次为：第 1 个插入的数，第 2 个插入的数，...第 n 个插入的数。

输入格式

第一行包含整数 M，表示操作次数。

接下来 M 行，每行包含一个操作命令，操作命令可能为以下几种：

1. `L x`，表示在链表的最左端插入数 x。
2. `R x`，表示在链表的最右端插入数 x。
3. `D k`，表示将第 k 个插入的数删除。
4. `IL k x`，表示在第 k 个插入的数左侧插入一个数。
5. `IR k x`，表示在第 k 个插入的数右侧插入一个数。

输出格式

共一行，将整个链表从左到右输出。

```
#include <iostream>

using namespace std;

const int N = 1e5 + 5;

int l[N], r[N], e[N];
// l表示该节点左边，r表示该节点右边
int idx;

void init() {
  l[1] = 0;
  r[0] = 1;      // 第一个点的右边是 1    第二个点的左边是 0，这两个点都会随着链表变化
  idx = 2; //已经用掉两个节点
}

// k点右边插入x
void add(int k, int x) {
  e[idx] = x;
  r[idx] = r[k];
  l[idx] = k;
  l[r[idx]] = idx;
```

```
  r[k] = idx;
  idx++;
}

void remove(int k) {
  r[l[k]] = r[k];
  l[r[k]] = l[k];
}

int main() {
  int m;
  cin >> m;
  init();
  while (m--) {
    string op;
    cin >> op;
    int k, x;
    if (op == "R") {
      cin >> x;
      add(l[1], x); //指向最右边的节点
    } else if (op == "L") {
      cin >> x;
      add(0, x); //指向最左边的节点
    } else if (op == "D") {
      cin >> k;
      remove(k + 1);
    } else if (op == "IL") {
      cin >> k >> x;
      add(l[k + 1], x); // k点右边插入
    } else if (op == "IR") {
      cin >> k >> x;
      add(k + 1, x);
    }
  }
  for (int i = r[0]; i != 1; i = r[i])
    cout << e[i] << ' ';
  return 0;
}
```

828.模拟栈

实现一个栈，栈初始为空，支持四种操作：

1. `push x` – 向栈顶插入一个数 x；
2. `pop` – 从栈顶弹出一个数；
3. `empty` – 判断栈是否为空；
4. `query` – 查询栈顶元素。

现在要对栈进行 M 个操作，其中的每个操作 3 和操作 4 都要输出相应的结果。

输入格式

第一行包含整数 M，表示操作次数。

接下来 M 行，每行包含一个操作命令，操作命令为 `push x`，`pop`，`empty`，`query` 中的一种。

输出格式

对于每个 `empty` 和 `query` 操作都要输出一个查询结果，每个结果占一行。

其中，`empty` 操作的查询结果为 `YES` 或 `NO`，`query` 操作的查询结果为一个整数，表示栈顶元素的值。

数据范围

$1 \leq M \leq 100000$,
$1 \leq x \leq 10^9$
所有操作保证合法。

```
#include <iostream>
using namespace std;
const int N = 100010;
int st[N];
int top = -1;
int n;


int main()
{
    cin >> n;
    while(n -- )
    {
        string s;
        cin >> s;
        if(s == "push")
        {
            cin >> st[++top];
        }
        if(s == "query")
        {
            cout << st[top] << endl;
        }
        if(s == "pop")
        {
            top--;
```

```
        }
        if(s == "empty")
        {
            if(top == -1)   cout << "YES" << endl;
            else            cout << "NO" << endl;
        }
    }
    return 0;
}
```

3302. 表达式求值

给定一个表达式，其中运算符仅包含 `+,-,*,/` （加 减 乘 整除），可能包含括号，请你求出表达式的最终值。

注意：

- 数据保证给定的表达式合法。
- 题目保证符号 `-` 只作为减号出现，不会作为负号出现，例如，`-1+2`，`(2+2)*(-(1+1)+2)` 之类表达式均不会出现。
- 题目保证表达式中所有数字均为正整数。
- 题目保证表达式在中间计算过程以及结果中，均不超过 $2^{31}-1$。
- 题目中的整除是指向 0 取整，也就是说对于大于 0 的结果向下取整，例如 $5/3=1$，对于小于 0 的结果向上取整，例如 $5/(1-4)=-1$。
- C++和Java中的整除默认是向零取整；Python中的整除 `//` 默认向下取整，因此Python的 `eval()` 函数中的整除也是向下取整，在本题中不能直接使用。

输入格式

共一行，为给定表达式。

输出格式

共一行，为表达式的结果。

数据范围

表达式的长度不超过 10^5。

输入样例：

```
(2+2)*(1+1)
```

输出样例：

```
8
```

```
#include <iostream>
#include <stack>
#include <string>
#include <unordered_map>
using namespace std;

stack<int> num;
stack<char> op;

unordered_map<char, int> h{{'+', 1}, {'-', 1}, {'*', 2}, {'/', 2}};
```

```

void eval() {
  int a = num.top();
  num.pop();
  int b = num.top();
  num.pop();
  char c = op.top();
  op.pop();
  int res = 0;
  if (c == '+')
    res = b + a;
  if (c == '-')
    res = b - a;
  if (c == '*')
    res = b * a;
  if (c == '/')
    res = b / a; //注意顺序

  num.push(res);
}

int main() {
  string s;
  cin >> s;
  int idx = 0;
  for (int i = 0; i < s.size(); i++) {
    if (isdigit(s[i])) {
      int j = i, res = 0;
      while (j < s.size() && isdigit(s[j])) {
        res = res * 10 + (s[j] - '0'); //可以不借助carry实现
        j++;
      }
      i = j - 1;
      num.push(res);
    } else if (s[i] == '(') {
      op.push('(');

    } else if (s[i] == ')') {
      while (op.top() != '(') {     //匹配到上一(
        eval();
      }
      op.pop(); //将'('出栈
    } else if (h.count(s[i])) {
      while (op.size() &&
             h[op.top()] >= h[s[i]]) //待入栈运算符优先级低，则先计算
        eval();                      //如 ***+ 的情况
      op.push(s[i]);                 //操作符入栈
    }
  }
  while (op.size())
    eval();                  //剩余的进行计算
  cout << num.top() << endl; //输出结果
  return 0;
```

```
}
```

829. 模拟队列

```
#include <iostream>

using namespace std;

const int N = 100005;
int q[N];
int hh = 0;
int tt = -1;

int main() {
  int n;
  string s;
  cin >> n;
  while (n--) {
    cin >> s;
    if (s == "push") {
      cin >> q[++tt];
    }
    if (s == "pop") {
      hh++;
    }
    if (s == "query") {
      cout << q[hh] << endl;
    }
    if (s == "empty") {
      if (hh > tt)
        cout << "YES" << endl;
      else
        cout << "NO" << endl;
    }
  }
}
```

830. 单调栈

给定一个长度为 N 的整数数列，输出每个数左边第一个比它小的数，如果不存在则输出 -1。

输入格式

第一行包含整数 N，表示数列长度。

第二行包含 N 个整数，表示整数数列。

输出格式

共一行，包含 N 个整数，其中第 i 个数表示第 i 个数的左边第一个比它小的数，如果不存在则输出 -1。

数据范围

$1 \le N \le 10^5$
$1 \le$ 数列中元素 $\le 10^9$

输入样例：

```
5
3 4 2 7 5
```

输出样例：

```
-1 3 -1 2 2
```

```cpp
#include <iostream>
#include <stack>

using namespace std;

stack<int> st;

int main() {
  int n, tmp;
  cin >> n;
  st.push(-1);
  while (n--) {
    cin >> tmp;

    while (tmp <= st.top())

      st.pop();

    cout << st.top() << " ";

    st.push(tmp);
  }
}
```

154 滑动窗口

给定一个大小为 $n \leq 10^6$ 的数组。

有一个大小为 k 的滑动窗口，它从数组的最左边移动到最右边。

你只能在窗口中看到 k 个数字。

每次滑动窗口向右移动一个位置。

以下是一个例子：

该数组为 `[1 3 -1 -3 5 3 6 7]`，k 为 3。

窗口位置	最小值	最大值
[1 3 -1] -3 5 3 6 7	-1	3
1 [3 -1 -3] 5 3 6 7	-3	3
1 3 [-1 -3 5] 3 6 7	-3	5
1 3 -1 [-3 5 3] 6 7	-3	5
1 3 -1 -3 [5 3 6] 7	3	6
1 3 -1 -3 5 [3 6 7]	3	7

你的任务是确定滑动窗口位于每个位置时，窗口中的最大值和最小值。

输入格式

输入包含两行。

第一行包含两个整数 n 和 k，分别代表数组长度和滑动窗口的长度。

第二行有 n 个整数，代表数组的具体数值。

同行数据之间用空格隔开。

输出格式

输出包含两个。

第一行输出，从左至右，每个位置滑动窗口中的最小值。

第二行输出，从左至右，每个位置滑动窗口中的最大值。

输入样例：

```
8 3
1 3 -1 -3 5 3 6 7
```

输出样例：

```
-1 -3 -3 -3 3 3
3 3 5 5 6 7
```

通过在队头维持当前窗口的最小值来实现滑动窗口。

```cpp
#include <iostream>
#include <cstring>
#include <algorithm>
#include <deque>
using namespace std;

const int N = 1000010;
int a[N];
int main()
{
    int n, k;
    cin >> n >> k;
    for (int i = 1; i <= n; i ++ ) cin >> a[i];//读入数据
    deque<int> q;
    for(int i = 1; i <= n; i++)
    {
        while(q.size() && q.back() > a[i]) //新进入窗口的值小于队尾元素，则队尾出队列
            q.pop_back();
        q.push_back(a[i]);//将新进入的元素入队
        if(i - k >= 1 && q.front() == a[i - k])//若队头是否滑出了窗口，队头出队
            q.pop_front();
        if(i >= k)//当窗口形成，输出队头对应的值
            cout << q.front() <<" ";
    }
    q.clear();
    cout << endl;

    //最大值亦然
    for(int i = 1; i <= n; i++)
    {
        while(q.size() && q.back() < a[i]) q.pop_back();
        q.push_back(a[i]);
        if(i - k >= 1 && a[i - k] == q.front()) q.pop_front();
        if(i >= k) cout << q.front() << " ";

    }
}

```

831. KMP字符串

给定一个字符串 S，以及一个模式串 P，所有字符串中只包含大小写英文字母以及阿拉伯数字。

模式串 P 在字符串 S 中多次作为子串出现。

求出模式串 P 在字符串 S 中所有出现的位置的起始下标。

输入格式

第一行输入整数 N，表示字符串 P 的长度。

第二行输入字符串 P。

第三行输入整数 M，表示字符串 S 的长度。

第四行输入字符串 S。

输出格式

共一行，输出所有出现位置的起始下标（下标从 0 开始计数），整数之间用空格隔开。

感觉这个记了没半小时就忘了ฅ_ฅ

```
#include <iostream>

using namespace std;

const int N = 100010, M = 1000010;

int n, m;
int ne[N];
char s[M], p[N];

int main()
{
    cin >> n >> p + 1 >> m >> s + 1;

    for (int i = 2, j = 0; i <= n; i ++ )
    {
        while (j && p[i] != p[j + 1]) j = ne[j];
        if (p[i] == p[j + 1]) j ++ ;
        ne[i] = j;
    }

    for (int i = 1, j = 0; i <= m; i ++ )
    {
        while (j && s[i] != p[j + 1]) j = ne[j];
        if (s[i] == p[j + 1]) j ++ ;
        if (j == n)
        {
            printf("%d ", i - n);
            j = ne[j];
        }
    }

```

```
    return 0;
}

```

Trie树(前缀树或字典树)

高效存储查找字符串

835. Trie字符串统计

维护一个字符串集合，支持两种操作：

1. `I x` 向集合中插入一个字符串 x；
2. `Q x` 询问一个字符串在集合中出现了多少次。

共有 N 个操作，所有输入的字符串总长度不超过 10^5，字符串仅包含小写英文字母。

输入格式

第一行包含整数 N，表示操作数。

接下来 N 行，每行包含一个操作指令，指令为 `I x` 或 `Q x` 中的一种。

输出格式

对于每个询问指令 `Q x`，都要输出一个整数作为结果，表示 x 在集合中出现的次数。

每个结果占一行。

```
#include <iostream>

using namespace std;

const int N = 100010;
//son[][]存储子节点的位置，分支最多26条；
//cnt[]存储以某节点结尾的字符串个数（同时也起标记作用）
//idx表示当前要插入的节点是第几个,每创建一个节点值+1
int son[N][32], cnt[N], idx;
char str[N];



void insert(char * str){
    int p = 0;      //节点1
    for(int i = 0; str[i]; i++){
        int u = str[i] - 'a';
        if(!son[p][u]) son[p][u] = ++idx;
        p = son[p][u];      //移动到子节点
    }
    cnt[p]++;           //以某节点结尾的字符串个数
}


```

```c
int query(char *str){
    int p = 0;
    for(int i = 0; str[i]; i++){
        int u = str[i] - 'a';
        if(!son[p][u]) return 0;
        p = son[p][u];
    }
    return cnt[p];
 }


int main()
{
    int m;
    cin >> m;

    while(m--)
    {
        char op[2];
        scanf("%s%s", op, str);

        if(*op == 'I') insert(str);
        else printf("%d\n", query(str));
    }

    return 0;
}
```

143. 最大异或对

在给定的 N 个整数 A_1，$A_2 \ldots\ldots A_N$ 中选出两个进行 xor（异或）运算，得到的结果最大是多少？

输入格式

第一行输入一个整数 N。

第二行输入 N 个整数 $A_1 \sim A_N$。

输出格式

输出一个整数表示答案。

数据范围

$1 \le N \le 10^5$,
$0 \le A_i < 2^{31}$

输入样例：

```
3
1 2 3
```

输出样例：

```
3
```

```
#include <iostream>
#include<algorithm>
using namespace std;
const int N = 100010;
int son[N*31][2], idx,n,a[N];

void insert(int x)
{
  int p = 0;
  for (int i = 30; i >= 0; i--)      //右移30才是第一位
  {
    int u = x >> i & 1;
    if (!son[p][u])
      son[p][u] = ++idx;
    p = son[p][u];
  }
}

int  query(int x)   //前x位相同的异或值
{
  int p = 0,res = 0;
  for (int i = 30; i >= 0; i--)
  {
    int u = x >> i & 1;
    if (son[p][!u])
    {
```

```
      p = son[p][!u];
      res = res * 2 + 1;
    }
    else
    {
      p = son[p][u];
      res = res * 2+0;
    }
  }
  return res;
}
int main()
{
  cin >> n;
  for (int i = 0; i < n; i++)
  {
    cin >> a[i];
    insert(a[i]);
  }
  int res = 0;
  for (int i = 0; i < n; i++)
  {
    res = max(res, query(a[i]));
  }
  cout << res;

  return 0;
}
```

并查集

- 将含有两个元素的两个集合合并
- 询问两个元素是否在一个集合中

优化：路径压缩

```
int find(int x) {
  if (p[x] != x)
    p[x] =find(p[x]);        //路径压缩
  return p[x];       //返回头节点
}
 for (int i = 1; i <= n; i++)
    p[i] = i;            //初始化并查集
```

acwing 836. 合并集合

一共有 n 个数，编号是 $1 \sim n$，最开始每个数各自在一个集合中。

现在要进行 m 个操作，操作共有两种：

1. `M a b`，将编号为 a 和 b 的两个数所在的集合合并，如果两个数已经在同一个集合中，则忽略这个操作；
2. `Q a b`，询问编号为 a 和 b 的两个数是否在同一个集合中；

输入格式

第一行输入整数 n 和 m。

接下来 m 行，每行包含一个操作指令，指令为 `M a b` 或 `Q a b` 中的一种。

输出格式

对于每个询问指令 `Q a b`，都要输出一个结果，如果 a 和 b 在同一集合内，则输出 `Yes`，否则输出 `No`。

每个结果占一行。

数据范围

$1 \le n, m \le 10^5$

输入样例：

```
4 5
M 1 2
M 3 4
Q 1 2
Q 1 3
Q 3 4
```

输出样例：

```
Yes
No
Yes
```

cpp

```cpp
#include <iostream>

const int N = 100050;
int n, m;
int p[N];

int find(int x) {
  if (p[x] != x)
    p[x] =find(p[x]);        //路径压缩
  return p[x];       //返回头节点
}

int main() {
  scanf("%d%d", &n, &m);
```

```c
  for (int i = 1; i <= n; i++)
    p[i] = i;           //将每个元素放入一个堆

  while (m--) {
    char op[2];
    int a, b;
    scanf("%s%d%d", op, &a, &b);
    if (op[0] == 'M')
      p[find(a)] = find(b);     //将a的头节点插入b
    else {
      if (find(a) == find(b))
        puts("Yes");
      else
        puts("No");
    }
  }
}
```

python

```python
import sys
input = lambda: sys.stdin.readline().strip()

n, m = map(int, input().split())

# 初始化
p = [i for i in range(n+1)]
size = [1] * (n+1)  # 按秩合并

# 非递归find
def find(x):
    while p[x] != x:
        p[x] = p[p[x]]  # 路径压缩
        x = p[x]
    return p[x]

for _ in range(m):
    op, *args = input().split()
    if op == "Q":
        a, b = map(int, args)
        print("Yes" if find(a) == find(b) else "No")
    elif op == "M":
        a, b = map(int, args)
        root_a = find(a)
        root_b = find(b)
        if root_a != root_b:
            if size[root_a] > size[root_b]:
                p[root_b] = root_a
                size[root_a] += size[root_b]
            else:
                p[root_a] = root_b
                size[root_b] += size[root_a]
```

837. 连通块中点的数量

给定一个包含 n 个点（编号为 $1 \sim n$）的无向图，初始时图中没有边。

现在要进行 m 个操作，操作共有三种：

1. `C a b`，在点 a 和点 b 之间连一条边，a 和 b 可能相等；
2. `Q1 a b`，询问点 a 和点 b 是否在同一个连通块中，a 和 b 可能相等；
3. `Q2 a`，询问点 a 所在连通块中点的数量；

输入格式

第一行输入整数 n 和 m。

接下来 m 行，每行包含一个操作指令，指令为 `C a b`，`Q1 a b` 或 `Q2 a` 中的一种。

输出格式

对于每个询问指令 `Q1 a b`，如果 a 和 b 在同一个连通块中，则输出 `Yes`，否则输出 `No`。

对于每个询问指令 `Q2 a`，输出一个整数表示点 a 所在连通块中点的数量

每个结果占一行。

数据范围

$1 \le n, m \le 10^5$

我们只需要再维持一个点的数量的数组。

cpp

```cpp
#include <iostream>
using namespace std;

const int N = 1000050;

int n, m;


int p[N], siz[N];


int find(int x){
    if(p[x] != x)   p[x] = find(p[x]);
    return p[x];    //p[x]已经被更新
}

int main()
{
    cin >> n >> m;
    for(int i = 1; i <= n; i++)
```

```c
    {
        p[i] = i;
        siz[i] = 1;
    }
    while(m--)

    {
        char op[5];
        scanf("%s", op);
        int a, b;
        if(op[0] == 'C')
        {
            scanf("%d%d", &a, &b);
            if(find(a) == find(b)) continue;    //同一连通图内点相连直接continue
            siz[find(b)] += siz[find(a)];
            p[find(a)] = find(b);
        }
        else if(op[1] == '2')
        {
            scanf("%d", &a);
            printf("%d\n", siz[find(a)]);
        }
        else
        {
            scanf("%d%d", &a, &b);
            if(find(a) == find(b))  puts("Yes");
            else puts("No");
        }


    }


}

```

python

```python
import sys
sys.setrecursionlimit(100000)
input = lambda: sys.stdin.readline().strip()

n, m = map(int, input().split())

p = [i for i in range(n+1)]
siz = [1] * (n+1)

def find(x):
    if p[x] != x:
        p[x] = find(p[x])
    return p[x]

for _ in range(m):
```

```python
    parts = input().split()
    op = parts[0]

    if op == "C":
        a, b = map(int, parts[1:])
        root_a = find(a)
        root_b = find(b)
        if root_a != root_b:
            p[root_a] = root_b
            siz[root_b] += siz[root_a]

    elif op == "Q1":
        a, b = map(int, parts[1:])
        print("Yes" if find(a) == find(b) else "No")

    elif op == "Q2":
        a = int(parts[1])
        print(siz[find(a)])
```

240. 食物链

动物王国中有三类动物 A,B,C，这三类动物的食物链构成了有趣的环形。

A 吃 B，B 吃 C，C 吃 A。

现有 N 个动物，以 $1\sim N$ 编号。

每个动物都是 A,B,C 中的一种，但是我们并不知道它到底是哪一种。

有人用两种说法对这 N 个动物所构成的食物链关系进行描述：

第一种说法是 `1 X Y`，表示 X 和 Y 是同类。

第二种说法是 `2 X Y`，表示 X 吃 Y。

此人对 N 个动物，用上述两种说法，一句接一句地说出 K 句话，这 K 句话有的是真的，有的是假的。

当一句话满足下列三条之一时，这句话就是假话，否则就是真话。

1. 当前的话与前面的某些真的话冲突，就是假话；
2. 当前的话中 X 或 Y 比 N 大，就是假话；
3. 当前的话表示 X 吃 X，就是假话。

你的任务是根据给定的 N 和 K 句话，输出假话的总数。

输入格式

第一行是两个整数 N 和 K，以一个空格分隔。

以下 K 行每行是三个正整数 D，X，Y，两数之间用一个空格隔开，其中 D 表示说法的种类。

若 $D=1$，则表示 X 和 Y 是同类。

若 $D=2$，则表示 X 吃 Y。

输出格式

只有一个整数，表示假话的数目。

数据范围

$1\le N\le 50000$,
$0\le K\le 100000$

输入样例：

```
100 7
1 101 1
2 1 2
2 2 3
2 3 3
1 1 3
2 3 1
1 5 5
```

输出样例：

```
3
```

- 余1可以吃根节点
- 余2可以被根节点吃

- 余3与根节点同类

//TODO

```cpp
#include <iostream>
using namespace std;

const int N = 5 * 1e4 + 10; // 定义最大节点数
int n, m; // n表示节点数，m表示操作数
int p[N], d[N]; // p数组存储每个节点的父节点，d数组存储每个节点到其父节点的距离

// 查找函数，带路径压缩和距离更新
int find(int x) {
    if (p[x] != x) {
        int t = find(p[x]); // t暂时存储x的父节点的根节点
        d[x] += d[p[x]]; // 更新x到根节点的距离
        p[x] = t; // 路径压缩
    }
    return p[x]; // 返回根节点
}

int main() {
    cin >> n >> m;
    // 初始化，每个节点的父节点为其自身
    for (int i = 1; i <= n; i++) {
        p[i] = i;
    }
    int res = 0; // 记录不合法操作的次数

    while (m--) {
        int r, x, y;
        cin >> r >> x >> y;

        // 检查输入是否合法
        if (x > n || y > n) {
            res++;
        } else {
            int px = find(x), py = find(y);

            // 处理r == 1的情况
            if (r == 1) {
                // 如果x和y在同一个集合中且不满足距离关系，则计数加1
                if (px == py && (d[x] - d[y]) % 3) {
                    res++;
                } else if (px != py) {
                    // 合并两个集合，更新距离关系
                    p[px] = py;
                    d[px] = d[y] - d[x];
                }
            }
            // 处理r == 2的情况
            else if (r == 2) {
                // 如果x和y在同一个集合中且不满足距离关系，则计数加1
```

```
                if (px == py && (d[x] - d[y] - 1) % 3) {
                    res++;
                } else if (px != py) {
                    // 合并两个集合，更新距离关系
                    p[px] = py;
                    d[px] = d[y] + 1 - d[x];
                }
            }
        }
    }
    cout << res; // 输出不合法操作的次数
}
```

P3958 [NOIP 2017 提高组] 奶酪

现有一块大奶酪，它的高度为 h，它的长度和宽度我们可以认为是无限大的，奶酪中间有许多半径相同的球形空洞。我们可以在这块奶酪中建立空间坐标系，在坐标系中，奶酪的下表面为 $z=0$，奶酪的上表面为 $z=h$。

现在，奶酪的下表面有一只小老鼠 Jerry，它知道奶酪中所有空洞的球心所在的坐标。如果两个空洞相切或是相交，则 Jerry 可以从其中一个空洞跑到另一个空洞，特别地，如果一个空洞与下表面相切或是相交，Jerry 则可以从奶酪下表面跑进空洞；如果一个空洞与上表面相切或是相交，Jerry 则可以从空洞跑到奶酪上表面。

位于奶酪下表面的 Jerry 想知道，在不破坏奶酪的情况下，能否利用已有的空洞跑 到奶酪的上表面去？

空间内两点 $P_1(x_1,y_1,z_1)$、$P2(x_2,y_2,z_2)$ 的距离公式如下：

$$\text{dist}(P_1,P_2)=\sqrt{(x_1-x_2)^2+(y_1-y_2)^2+(z_1-z_2)^2}$$

输入格式

每个输入文件包含多组数据。

第一行，包含一个正整数 T，代表该输入文件中所含的数据组数。

接下来是 T 组数据，每组数据的格式如下： 第一行包含三个正整数 n,h,r，两个数之间以一个空格分开，分别代表奶酪中空洞的数量，奶酪的高度和空洞的半径。

接下来的 n 行，每行包含三个整数 x,y,z，两个数之间以一个空格分开，表示空洞球心坐标为 (x,y,z)。

输出格式

T 行，分别对应 T 组数据的答案，如果在第 i 组数据中，Jerry 能从下表面跑到上表面，则输出 `Yes`，如果不能，则输出 `No`。

输入输出样例

输入 #1 复制

```
3
2 4 1
0 0 1
0 0 3
2 5 1
0 0 1
0 0 4
2 5 2
0 0 2
2 0 4
```

输出 #1 复制

```
Yes
No
Yes
```

这题的关键是识别出这是属于并查集的答案， 同时注意对于大数据long long类型的比较

思路：

1. 将每个洞虚拟化为并查集集合，并记录连接底部和顶部的洞。
2. 双重循环，遍历合并能连通的洞

3. 判断连接底部和顶部的点是否位于同一集合

```
#include <iostream>
#include <utility>
using namespace std;
typedef long long LL;
const int N = 100001;

typedef pair<int, int> PA;
typedef pair<PA, int> PA2;


int T;
int n,h,r;
long long point[N][3];
int toppoint[N], lowpoint[N]; //记录最高最低点
int p[N];


int lowcnt = 0;
int topcnt = 0;


int find(int x){
    if(p[x] != x)
        p[x] = find(p[x]);
    return p[x];
}



bool distin(int i, int j)
{
    return  4LL*r*r >=  ((point[i][0]-point[j][0])*(point[i][0]-point[j][0])
     + (point[i][1]-point[j][1])* (point[i][1]-point[j][1])
     +(point[i][2]-point[j][2])*(point[i][2]-point[j][2]) ) ? true : false;
}




void solve() {
    lowcnt = 0;
    topcnt = 0;
    cin >> n >> h >> r;



    for(int i = 1; i <= n; i++)
        p[i] = i;

    for(int i = 1; i <=n; i++)
    {
```

```
        cin >> point[i][0] >>point[i][1]  >> point[i][2] ;
        if(point[i][2] - r <= 0)  lowpoint[lowcnt++] = i;
        if(point[i][2]+r >= h)   toppoint[topcnt++] = i;
    }
    for(int i = 1; i <= n; i++)
    {
        for(int j = i+1; j <= n; j++)
        {
                if(distin(i,j))
                    p[find(j)] = find(i);
                    // int x = find(i);
                    // int y = find(j);
                    // if (x != y) p[x] = y;
        }
    }
    for(int i = 0; i < topcnt;i++)
    {
        for(int j = 0; j < lowcnt; j++)
        {
                if(find(toppoint[i]) == find(lowpoint[j]))
                {
                    cout << "Yes" << endl;
                    return ;
                }
        }
    }
    cout << "No" << endl;

}

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> T;
    while(T--) {
       solve();
    }
}

```

堆

- 插入一个数
- 求当前集合最小值
- 删除最小值
- 删除或修改任意元素

- 1为根，左儿子，2x，右儿子2x+1

838. 堆排序

输入一个长度为 n 的整数数列，从小到大输出前 m 小的数。

输入格式

第一行包含整数 n 和 m。

第二行包含 n 个整数，表示整数数列。

输出格式

共一行，包含 m 个整数，表示整数数列中前 m 小的数。

数据范围

$1 \le m \le n \le 10^5$，
$1 \le$ 数列中元素 $\le 10^9$

输入样例:

```
5 3
4 5 1 3 2
```

输出样例:

```
1 2 3
```

```cpp
#include <iostream>

using namespace std;
int n, m, siz;
const int N = 100010;

int h[N];
void down(int u) {
  int t = u;
  if (h[2 * u] < h[t] && 2 * u <= siz)
    t = 2 * u;
  if (h[2 * u + 1] < h[t] && (2 * u + 1) <= siz)
    t = 2 * u + 1;
  if (t != u) {
    swap(h[t], h[u]);
    //别忘了递归
    down(t);
  }
}
int main() {
  cin >> n >> m;
  for (int i = 1; i <= n; i++)
    cin >> h[i]; //从1开始
```

```

  siz = n; // 将 siz 初始化为 n       要在down前初始化

  for (int i = n / 2; i >= 1; i--)
    down(i);

  for (int i = 0; i < m; i++) {
    cout << h[1] << " ";
    h[1] = h[siz--];

    down(1);
  }
}
```

839. 模拟堆

维护一个集合，初始时集合为空，支持如下几种操作：

1. `I x`，插入一个数 x；
2. `PM`，输出当前集合中的最小值；
3. `DM`，删除当前集合中的最小值（数据保证此时的最小值唯一）；
4. `D k`，删除第 k 个插入的数；
5. `C k x`，修改第 k 个插入的数，将其变为 x；

现在要进行 N 次操作，对于所有第 2 个操作，输出当前集合的最小值。

输入格式

第一行包含整数 N。

接下来 N 行，每行包含一个操作指令，操作指令为 `I x`，`PM`，`DM`，`D k` 或 `C k x` 中的一种。

输出格式

对于每个输出指令 `PM`，输出一个结果，表示当前集合中的最小值。

每个结果占一行。

数据范围

$1 \le N \le 10^5$
$-10^9 \le x \le 10^9$
数据保证合法。

```
#include<iostream>
#include<algorithm>
using namespace std;

const int N=1e5+10;
int h[N];   //堆
int ph[N];  //存放第k个插入点的下标
int hp[N];  //存放堆中点的插入次序
int cur_size;   //size 记录的是堆当前的数据多少,即堆的最后一个元素
```

```cpp
void heap_swap(int a,int b)
{
    swap(h[a],h[b]);            //交换堆中两个元素的值
    swap(hp[a],hp[b]);          //交换插入次序
    swap(ph[hp[a]],ph[hp[b]])  ;   //交换插入点的下标
}



void down(int u)
{
    int t=u;
    if(u*2<=cur_size&&h[t]>h[u*2]) t=u*2;
    if(u*2+1<=cur_size&&h[t]>h[u*2+1])  t=u*2+1;
    if(u!=t)
    {
        heap_swap(u,t);
        down(t);
    }
}
void up(int u)
{
    if(u/2>0&&h[u]<h[u/2])
    {
        heap_swap(u,u/2);
        up(u>>1);
    }
}




int main()
{
    int n;
    cin >> n;
    int m = 0;        //m用来记录插入的数的个数
      while(n--)
    {
        string op;
        int k,x;
        cin>>op;
        if(op=="I")
        {
            cin>>x;
            m++;
            h[++cur_size]=x;    //置于堆的最后
            ph[m]=cur_size;     //记录第k个插入点的下标
```

```
            hp[cur_size]=m;      //记录插入次序
            //down(size);
            up(cur_size);
        }
        else if(op=="PM")    cout<<h[1]<<endl;
        else if(op=="DM")
        {
            heap_swap(1,cur_size);
            cur_size--;
            down(1);
        }
        else if(op=="D")
        {
            cin>>k;
            int u=ph[k];                //这里一定要用u=ph[k]保存第k个插入点的下标
            heap_swap(u,cur_size);          //因为在此处heap_swap操作后ph[k]的值已经发生
            cur_size--;                    //如果在up,down操作中仍然使用ph[k]作为参数就会发生错误
            up(u);
           down(u);
        }
        else if(op=="C")
        {
            cin>>k>>x;
            h[ph[k]]=x;                 //此处由于未涉及heap_swap操作且下面的up、down操作只会发生一个所以
            down(ph[k]);                //所以可直接传入ph[k]作为参数
            up(ph[k]);
        }

    }
    return 0;


}

```

//TODO

哈希表

840. 模拟散列表

维护一个集合，支持如下几种操作：

1. `I x`，插入一个整数 x；
2. `Q x`，询问整数 x 是否在集合中出现过；

现在要进行 N 次操作，对于每个询问操作输出对应的结果。

输入格式

第一行包含整数 N，表示操作数量。

接下来 N 行，每行包含一个操作指令，操作指令为 `I x`，`Q x` 中的一种。

输出格式

对于每个询问指令 `Q x`，输出一个询问结果，如果 x 在集合中出现过，则输出 `Yes`，否则输出 `No`。

每个结果占一行。

拉链法：

数组 +（重复元素插入链表）

```cpp
#include <cstring>
#include <iostream>

using namespace std;

const int N = 100003;

int h[N], e[N], ne[N], idx;

void insert(int x) //链式前向星
{
  int k = (x % N + N) % N;
  //(x + N)%N;  不能采用以下方式因为x可能远大于N的范围
  e[idx] = x;      //插入元素
  ne[idx] = h[k]; //将插入的元素插入到链表的头部
  h[k] = idx++;    //指向最后一个插入的元素
}

bool find(int x) {
  int pos = (x % N + N) % N;
  for (int i = h[pos]; i != -1; i = ne[i]) {
    if (x == e[i])
      return true;
  }
  return false;
}

int main() {
  memset(h, -1, sizeof(h));
```

```
  int n;
  scanf("%d", &n);
  while (n--) {
    char op[2];
    int x;
    scanf("%s%d", op, &x);
    if (op[0] == 'I')
      insert(x);
    else if (op[0] == 'Q') {
      if (find(x))
        printf("Yes\n");
      else
        printf("No\n");
    }
  }
}
```

开放寻址法

N应该取题目要求的两到三倍

```
 (x % N + N) % N;
第一个x % N可能得到负数(如果x为负)
+ N确保结果为正
第二个% N确保结果在正确范围内
```

cpp

```
#include <cstring>
#include <iostream>

using namespace std;

const int N = 200003, null = 0x3f3f3f3f;
//0x3f3f3f3f * 2 = 2122219134，无穷大相加依然不会溢出。

int h[N];

int find(int x) {
  int pos = (x % N + N) % N;
  while(h[pos] != null && h[pos] != x)
  {
    pos++;
    if(pos == N)
      pos = 0;
  }
  return pos;
}

```

```c
int main() {
  memset(h, 0x3f, sizeof(h));
  int n;
  scanf("%d", &n);
  while (n--) {
    char op[2];
    int x;
    scanf("%s%d", op, &x);
    if (op[0] == 'I')
        h[find(x)] = x;
    else if (op[0] == 'Q') {
      if (h[find(x)] != null)
        printf("Yes\n");
      else
        printf("No\n");
    }
  }
}
```

python

```python
n = int(input())

null = 0x3f3f3f3f
arr =  [null]*n*3


# 返回第一个满足以下条件的位置：
# 槽位为空(h[pos] == null)，可以插入新元素
# 槽位已包含x(h[pos] == x)，表示元素已存在
def find(x):
    pos = (x%n+n)%n
    while arr[pos] !=null and  arr[pos] != x:
        pos = pos+1
        if pos == n:
            pos = 0
    return pos

for i in range(n):
    op, x = input().split()
    x = int(x)
    if op == 'I':
        arr[find(x)] = x
    else:
        pos = find(x)
        if arr[pos] != null:
            print('Yes')
        else:
            print('No')
```

AcWing 841. 字符串哈希（字符串前缀哈希法）

给定一个长度为 n 的字符串，再给定 m 个询问，每个询问包含四个整数 l_1, r_1, l_2, r_2，请你判断 $[l_1, r_1]$ 和 $[l_2, r_2]$ 这两个区间所包含的字符串子串是否完全相同。

字符串中只包含大小写英文字母和数字。

输入格式

第一行包含整数 n 和 m，表示字符串长度和询问次数。

第二行包含一个长度为 n 的字符串，字符串中只包含大小写英文字母和数字。

接下来 m 行，每行包含四个整数 l_1, r_1, l_2, r_2，表示一次询问所涉及的两个区间。

注意，字符串的位置从 1 开始编号。

输出格式

对于每个询问输出一个结果，如果两个字符串子串完全相同则输出 `Yes`，否则输出 `No`。

每个结果占一行。

数据范围

$1 \le n, m \le 10^5$

[AcWing 841. 字符串哈希 【公式助理解】 - AcWing]

cpp

P = 13331,Q=2^64

- 创建方程
  - h[i] = [h[i-1]*P+ ord(arr[i-1])] %mod, 创建哈希 （注意下标，防止查询时数组越界）
  - p[i] = p[i]*P % mod p`*`i次方
- 查询方程h[r] - h[l-1] * p[r-l+1] l-1>=0

```cpp
#include<iostream>
#include <string>
using namespace std;
typedef unsigned long long ULL;
const int N = 1e5+5,P = 13331;
ULL h[N],p[N];

ULL query(int l,int r){
    return h[r] - h[l - 1]*p[r - l + 1];
}
int main()
{
    int n, m;
    cin >> n >> m;
    string s;
    cin >> s;
```

```
    p[0] = 1;
    h[0] = 0;
    for(int i =0 ; i < s.size(); i++)
    {
        h[i + 1] = h[i] * P + s[i];
        p[i + 1] = p[i] * P;
    }
    while(m--)
    {
        int l1, l2, r1, r2;
        cin >> l1 >> r1 >> l2 >> r2;
        if(query(l1, r1) == query(l2, r2))
            cout << "Yes" << endl;
        else
            cout << "No" << endl;
    }

}

```

python

```
n,m = map(int,input().split())

mod = 10**9+7
P = 13331

h = [0]*(n+10)
p = [1]*(n+10)

def query(l,r):
    return (h[r] - h[l-1]*p[r-l+1])%mod     #求区间哈希值,pyhon无法利用ull性质，需要手动取模。


s = input()

for i in range(1,n+1):  #从第一位开始预处理
    p[i] = (p[i-1]*P) % mod  #预处理p的i次方
    h[i] = (h[i-1]*P + ord(s[i-1])) % mod #字符串哈希


for _ in range(m):
    l1,r1,l2,r2 = map(int,input().split())
    if query(l1,r1) == query(l2,r2):
        print('Yes')
    else:
        print('No')
```